

SATQUERY AI

P2 — Kedar

COMPLETE ENGINEERING PRD

AI / Agent Engineer

Deadline: 20 September 2026 • Six developers in parallel • Contract-first microservices

This is an execution document, not an API-only specification. It defines ownership, repository locations, modules, issues, dependencies, tests, security, observability, daily execution and a ready-to-copy coding-agent prompt.

1. Role Mission

Field	Implementation specification
Primary owner	LangGraph, Mission Copilot, intent, tools, sensor arbitration, Evidence/Confidence
Backup/review	Evidence; AI regression
Architecture reviewer	P1
QA/DevOps	P6
Success condition	Owned subsystem is independently runnable, contract-conformant, observable, secure and integrated into the flagship flood mission.
Non-ownership	Other teams' private tables, internal implementation details, credentials and undocumented contract changes.

2. System Position

P2 system position



Communication: browser→Gateway uses HTTPS/WebSocket; service-to-service uses versioned APIs/events; long-running EO/ML/report/monitoring work is asynchronous; storage and schemas have explicit ownership.

3. Repository / Folder Ownership

Folder	Purpose	Must not contain
services/agent/app/api/	Owned implementation, tests or docs	Unrelated service logic / credentials
graph/	Owned implementation, tests or docs	Unrelated service logic / credentials
nodes/	Owned implementation, tests or docs	Unrelated service logic / credentials
tools/	Owned implementation, tests or docs	Unrelated service logic / credentials
evidence/	Owned implementation, tests or docs	Unrelated service logic / credentials
security/	Owned implementation, tests or docs	Unrelated service logic / credentials
evals/agent/	Owned implementation, tests or docs	Unrelated service logic / credentials
tests/	Owned implementation, tests or docs	Unrelated service logic / credentials

4. Technology Stack

Field	Implementation specification
Stack	Python/LangGraph/Pydantic/Redis-Celery/LLM-VLM tool registry/OpenTelemetry
Why	Matches the supplied SatQuery architecture and the six-person playbook; additions are implementation recommendations, not product requirements.
Serialization	OpenAPI/JSON Schema; canonical DTOs/events.
Testing	pytest and contract/integration tests; frontend uses browser E2E where applicable.
Observability	OpenTelemetry + structured logs + Prometheus/Grafana.
Containerization	Docker; Kubernetes remains future/post-SIH.

5. Engineering Rules

- All internal APIs use /api/v1; breaking changes require versioning and P1 review.
- Canonical ErrorResponse: code, message, details[], retryable, trace_id.
- Propagate trace_id, mission_id, run_id, job_id, organization_id where applicable.
- organization_id comes from verified auth context, never a request body.
- No cross-service private DB reads.
- Long-running EO/ML/report/monitoring work returns 202 + job_id.

- Results retain dataset_id/observation_ids/model_version/processing_version as applicable.
- Idempotency-Key is used for duplicate-sensitive writes.
- Remote calls use bounded timeouts/retries/backoff; no retry storms.
- Never log secrets, auth headers or raw sensitive inputs.

6. Complete Issue Register

ID	Title	Pri	Type	Depends	Contact
P2-01	Agent service skeleton and MissionState	P0	Infrastructure	P1-01	P1
P2-02	Input sanitizer and intent validator	P0	Security	P1-01,P1-02	P4
P2-03	Intent extraction and mission planning	P0	Feature	P1-01,P1-02	P3
P2-04	Async agent execute endpoint and run orchestration	P0	Integration	P2-03	P1
P2-05	Versioned tool registry and schemas	P0	Feature/Security	P2-04	P4
P2-06	Permission-aware tool executor and budgets	P0	Security	P2-05	P3
P2-07	Observation search and asset selection tools	P0	Integration	P2-06,P1-02	P1
P2-08	Temporal planning and previous/current selection	P0	Feature	P2-07,P1-02	P4
P2-09	Adaptive sensor arbitration	P0	Feature	P2-08,P1-02	P3
P2-10	Evidence object / graph builder	P0	Feature	P2-09,P1-02	P1
P2-11	Confidence and uncertainty gate	P0	Safety/Feature	P2-10	P4
P2-12	Sensor disagreement analysis	P1	Feature	P2-11	P3
P2-13	Autonomous evidence acquisition loop	P1	Feature	P2-12	P1
P2-14	Evidence-only output synthesizer and WHY explanation	P0	Feature	P2-13	P4
P2-15	Prompt/evaluation harness and agent regression	P0	Testing	P2-14,P4-05	P3
P2-16	Agent observability and tool-call audit	P0	Observability	P2-15,P4-05	P1
P2-17	Agent failure/recovery policies	P1	Reliability	P2-16,P4-05	P4
P2-18	Agent release hardening and pinned demo profile	P0	Release	P2-17,P4-05	P3

P2-01 — Agent service skeleton and MissionState

Field	Implementation specification
Priority	P0
Type	Infrastructure
Owner	P2
Reviewer	P1
Cross-service contact	P1
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement agent service skeleton and missionstate as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for agent service skeleton and missionstate. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_agent_service_skeleton_and_missionstate_valid()', 'test_agent_service_skeleton_and_missionstate_invalid_input()']
Integration tests	['test_p2_01_service_boundary()']
Contract tests	['test_p2_01_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, agent service skeleton and missionstate completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-02
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run agent service skeleton and missionstate against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-02 — Input sanitizer and intent validator

Field	Implementation specification
Priority	P0
Type	Security
Owner	P2
Reviewer	P1
Cross-service contact	P4
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement input sanitizer and intent validator as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for input sanitizer and intent validator. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_input_sanitizer_and_intent_validator_valid()', 'test_input_sanitizer_and_intent_validator_invalid_input()']
Integration tests	['test_p2_02_service_boundary()']
Contract tests	['test_p2_02_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, input sanitizer and intent validator completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-03
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run input sanitizer and intent validator against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-03 — Intent extraction and mission planning

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P2
Reviewer	P1
Cross-service contact	P3
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement intent extraction and mission planning as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for intent extraction and mission planning. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_intent_extraction_and_mission_planning_valid()', 'test_intent_extraction_and_mission_planning_invalid_input()']
Integration tests	['test_p2_03_service_boundary()']
Contract tests	['test_p2_03_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, intent extraction and mission planning completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-04
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run intent extraction and mission planning against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-04 — Async agent execute endpoint and run orchestration

Field	Implementation specification
Priority	P0
Type	Integration
Owner	P2
Reviewer	P1
Cross-service contact	P1
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement async agent execute endpoint and run orchestration as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for async agent execute endpoint and run orchestration. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_async_agent_execute_endpoint_and_run_orchestration_valid()', 'test_async_agent_execute_endpoint_and_run_orchestration_invalid_input()']
Integration tests	['test_p2_04_service_boundary()']
Contract tests	['test_p2_04_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, async agent execute endpoint and run orchestration completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-03
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-05
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run async agent execute endpoint and run orchestration against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-05 — Versioned tool registry and schemas

Field	Implementation specification
Priority	P0
Type	Feature/Security
Owner	P2
Reviewer	P1
Cross-service contact	P4
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement versioned tool registry and schemas as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for versioned tool registry and schemas. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_versioned_tool_registry_and_schemas_valid()', 'test_versioned_tool_registry_and_schemas_invalid_input()']
Integration tests	['test_p2_05_service_boundary()']
Contract tests	['test_p2_05_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, versioned tool registry and schemas completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-04
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-06
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run versioned tool registry and schemas against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-06 — Permission-aware tool executor and budgets

Field	Implementation specification
Priority	P0
Type	Security
Owner	P2
Reviewer	P1
Cross-service contact	P3
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement permission-aware tool executor and budgets as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for permission-aware tool executor and budgets. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_permission_aware_tool_executor_and_budgets_valid()', 'test_permission_aware_tool_executor_and_budgets_invalid_input()']
Integration tests	['test_p2_06_service_boundary()']
Contract tests	['test_p2_06_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, permission-aware tool executor and budgets completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-07
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run permission-aware tool executor and budgets against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-07 — Observation search and asset selection tools

Field	Implementation specification
Priority	P0
Type	Integration
Owner	P2
Reviewer	P1
Cross-service contact	P1
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement observation search and asset selection tools as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for observation search and asset selection tools. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_observation_search_and_asset_selection_tools_valid()', 'test_observation_search_and_asset_selection_tools_invalid_input()']
Integration tests	['test_p2_07_service_boundary()']
Contract tests	['test_p2_07_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, observation search and asset selection tools completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-06,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-08
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run observation search and asset selection tools against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-08 — Temporal planning and previous/current selection

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P2
Reviewer	P1
Cross-service contact	P4
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement temporal planning and previous/current selection as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for temporal planning and previous/current selection. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_temporal_planning_and_previous_current_selection_valid()', 'test_temporal_planning_and_previous_current_selection_invalid_input()']
Integration tests	['test_p2_08_service_boundary()']
Contract tests	['test_p2_08_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, temporal planning and previous/current selection completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-07,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-09
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run temporal planning and previous/current selection against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-09 — Adaptive sensor arbitration

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P2
Reviewer	P1
Cross-service contact	P3
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement adaptive sensor arbitration as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for adaptive sensor arbitration. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_adaptive_sensor_arbitration_valid()', 'test_adaptive_sensor_arbitration_invalid_input()']
Integration tests	['test_p2_09_service_boundary()']
Contract tests	['test_p2_09_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, adaptive sensor arbitration completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-08,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-10
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run adaptive sensor arbitration against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-10 — Evidence object / graph builder

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P2
Reviewer	P1
Cross-service contact	P1
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement evidence object / graph builder as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for evidence object / graph builder. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_evidence_object_graph_builder_valid()', 'test_evidence_object_graph_builder_invalid_input()']
Integration tests	['test_p2_10_service_boundary()']
Contract tests	['test_p2_10_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, evidence object / graph builder completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-09,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-11
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run evidence object / graph builder against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-11 — Confidence and uncertainty gate

Field	Implementation specification
Priority	P0
Type	Safety/Feature
Owner	P2
Reviewer	P1
Cross-service contact	P4
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement confidence and uncertainty gate as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for confidence and uncertainty gate. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_confidence_and_uncertainty_gate_valid()', 'test_confidence_and_uncertainty_gate_invalid_input()']
Integration tests	['test_p2_11_service_boundary()']
Contract tests	['test_p2_11_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, confidence and uncertainty gate completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-10
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-12
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run confidence and uncertainty gate against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-12 — Sensor disagreement analysis

Field	Implementation specification
Priority	P1
Type	Feature
Owner	P2
Reviewer	P1
Cross-service contact	P3
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement sensor disagreement analysis as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for sensor disagreement analysis. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_sensor_disagreement_analysis_valid()', 'test_sensor_disagreement_analysis_invalid_input()']
Integration tests	['test_p2_12_service_boundary()']
Contract tests	['test_p2_12_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, sensor disagreement analysis completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-11
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-13
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run sensor disagreement analysis against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-13 — Autonomous evidence acquisition loop

Field	Implementation specification
Priority	P1
Type	Feature
Owner	P2
Reviewer	P1
Cross-service contact	P1
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement autonomous evidence acquisition loop as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for autonomous evidence acquisition loop. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/Internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_autonomous_evidence_acquisition_loop_valid()', 'test_autonomous_evidence_acquisition_loop_invalid_input()']
Integration tests	['test_p2_13_service_boundary()']
Contract tests	['test_p2_13_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, autonomous evidence acquisition loop completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-12
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-14
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run autonomous evidence acquisition loop against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-14 — Evidence-only output synthesizer and WHY explanation

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P2
Reviewer	P1
Cross-service contact	P4
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement evidence-only output synthesizer and why explanation as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for evidence-only output synthesizer and why explanation. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_evidence_only_output_synthesizer_and_why_explanation_valid()', 'test_evidence_only_output_synthesizer_and_why_explanation_invalid_input()']
Integration tests	['test_p2_14_service_boundary()']
Contract tests	['test_p2_14_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, evidence-only output synthesizer and why explanation completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-13
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-15
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run evidence-only output synthesizer and why explanation against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-15 — Prompt/evaluation harness and agent regression

Field	Implementation specification
Priority	P0
Type	Testing
Owner	P2
Reviewer	P1
Cross-service contact	P3
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement prompt/evaluation harness and agent regression as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for prompt/evaluation harness and agent regression. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_prompt_evaluation_harness_and_agent_regression_valid()', 'test_prompt_evaluation_harness_and_agent_regression_invalid_input()']
Integration tests	['test_p2_15_service_boundary()']
Contract tests	['test_p2_15_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, prompt/evaluation harness and agent regression completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-14, P4-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-16
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run prompt/evaluation harness and agent regression against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-16 — Agent observability and tool-call audit

Field	Implementation specification
Priority	P0
Type	Observability
Owner	P2
Reviewer	P1
Cross-service contact	P1
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement agent observability and tool-call audit as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for agent observability and tool-call audit. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_agent_observability_and_tool_call_audit_valid()', 'test_agent_observability_and_tool_call_audit_invalid_input()']
Integration tests	['test_p2_16_service_boundary()']
Contract tests	['test_p2_16_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, agent observability and tool-call audit completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-15,P4-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-17
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run agent observability and tool-call audit against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-17 — Agent failure/recovery policies

Field	Implementation specification
Priority	P1
Type	Reliability
Owner	P2
Reviewer	P1
Cross-service contact	P4
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement agent failure/recovery policies as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for agent failure/recovery policies. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_agent_failure_recovery_policies_valid()', 'test_agent_failure_recovery_policies_invalid_input()']
Integration tests	['test_p2_17_service_boundary()']
Contract tests	['test_p2_17_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, agent failure/recovery policies completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-16,P4-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P2-18
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run agent failure/recovery policies against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P2-18 — Agent release hardening and pinned demo profile

Field	Implementation specification
Priority	P0
Type	Release
Owner	P2
Reviewer	P1
Cross-service contact	P3
Service	Agent
Folders	services/agent/app/api/; graph/; nodes/; tools/; evidence/; security/; evals/agent/; tests/
Objective	Implement agent release hardening and pinned demo profile as a production-shaped module inside the P2 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for agent release hardening and pinned demo profile. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/agent/app/api/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/agent/plan, /api/v1/agent/execute, /api/v1/agent/sensor-decision, /api/v1/agent/confidence
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_agent_release_hardening_and_pinned_demo_profile_valid()', 'test_agent_release_hardening_and_pinned_demo_profile_invalid_input()']
Integration tests	['test_p2_18_service_boundary()']
Contract tests	['test_p2_18_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, agent release hardening and pinned demo profile completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P2-17, P4-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	Release/integration gate
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P2. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run agent release hardening and pinned demo profile against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

7. Test Plan

The issue-level test names below are the minimum required suite. Expand edge cases discovered during implementation.

P2-01 — Agent service skeleton and MissionState

- test_agent_service_skeleton_and_missionstate_valid()
- test_agent_service_skeleton_and_missionstate_invalid_input()
- test_p2_01_service_boundary()
- test_p2_01_schema_compatibility()

P2-02 — Input sanitizer and intent validator

- test_input_sanitizer_and_intent_validator_valid()
- test_input_sanitizer_and_intent_validator_invalid_input()
- test_p2_02_service_boundary()
- test_p2_02_schema_compatibility()

P2-03 — Intent extraction and mission planning

- test_intent_extraction_and_mission_planning_valid()
- test_intent_extraction_and_mission_planning_invalid_input()
- test_p2_03_service_boundary()
- test_p2_03_schema_compatibility()

P2-04 — Async agent execute endpoint and run orchestration

- test_async_agent_execute_endpoint_and_run_orchestration_valid()
- test_async_agent_execute_endpoint_and_run_orchestration_invalid_input()
- test_p2_04_service_boundary()
- test_p2_04_schema_compatibility()

P2-05 — Versioned tool registry and schemas

- test_versioned_tool_registry_and_schemas_valid()
- test_versioned_tool_registry_and_schemas_invalid_input()
- test_p2_05_service_boundary()
- test_p2_05_schema_compatibility()

P2-06 — Permission-aware tool executor and budgets

- test_permission_aware_tool_executor_and_budgets_valid()
- test_permission_aware_tool_executor_and_budgets_invalid_input()
- test_p2_06_service_boundary()
- test_p2_06_schema_compatibility()

P2-07 — Observation search and asset selection tools

- test_observation_search_and_asset_selection_tools_valid()
- test_observation_search_and_asset_selection_tools_invalid_input()
- test_p2_07_service_boundary()
- test_p2_07_schema_compatibility()

P2-08 — Temporal planning and previous/current selection

- test_temporal_planning_and_previous_current_selection_valid()
- test_temporal_planning_and_previous_current_selection_invalid_input()
- test_p2_08_service_boundary()
- test_p2_08_schema_compatibility()

P2-09 — Adaptive sensor arbitration

- test_adaptive_sensor_arbitration_valid()
- test_adaptive_sensor_arbitration_invalid_input()
- test_p2_09_service_boundary()
- test_p2_09_schema_compatibility()

P2-10 — Evidence object / graph builder

- test_evidence_object_graph_builder_valid()
- test_evidence_object_graph_builder_invalid_input()
- test_p2_10_service_boundary()
- test_p2_10_schema_compatibility()

P2-11 — Confidence and uncertainty gate

- test_confidence_and_uncertainty_gate_valid()
- test_confidence_and_uncertainty_gate_invalid_input()
- test_p2_11_service_boundary()
- test_p2_11_schema_compatibility()

P2-12 — Sensor disagreement analysis

- test_sensor_disagreement_analysis_valid()
- test_sensor_disagreement_analysis_invalid_input()
- test_p2_12_service_boundary()
- test_p2_12_schema_compatibility()

P2-13 — Autonomous evidence acquisition loop

- test_autonomous_evidence_acquisition_loop_valid()
- test_autonomous_evidence_acquisition_loop_invalid_input()
- test_p2_13_service_boundary()
- test_p2_13_schema_compatibility()

P2-14 — Evidence-only output synthesizer and WHY explanation

- test_evidence_only_output_synthesizer_and_why_explanation_valid()
- test_evidence_only_output_synthesizer_and_why_explanation_invalid_input()
- test_p2_14_service_boundary()
- test_p2_14_schema_compatibility()

P2-15 — Prompt/evaluation harness and agent regression

- test_prompt_evaluation_harness_and_agent_regression_valid()
- test_prompt_evaluation_harness_and_agent_regression_invalid_input()
- test_p2_15_service_boundary()
- test_p2_15_schema_compatibility()

P2-16 — Agent observability and tool-call audit

- test_agent_observability_and_tool_call_audit_valid()
- test_agent_observability_and_tool_call_audit_invalid_input()
- test_p2_16_service_boundary()
- test_p2_16_schema_compatibility()

P2-17 — Agent failure/recovery policies

- test_agent_failure_recovery_policies_valid()
- test_agent_failure_recovery_policies_invalid_input()
- test_p2_17_service_boundary()
- test_p2_17_schema_compatibility()

P2-18 — Agent release hardening and pinned demo profile

- test_agent_release_hardening_and_pinned_demo_profile_valid()
- test_agent_release_hardening_and_pinned_demo_profile_invalid_input()
- test_p2_18_service_boundary()
- test_p2_18_schema_compatibility()

8. Security Implementation

Control	Implementation owner	Required behavior
Authentication	P2	Implement/test where applicable; P6 supplies regression gate.
RBAC + tenant isolation	P2	Implement/test where applicable; P6 supplies regression gate.
Input/schema validation	P2	Implement/test where applicable; P6 supplies regression gate.
GeoJSON/raster validation	P2	Implement/test where applicable; P6 supplies regression gate.
SSRF/provider allowlist	P2	Implement/test where applicable; P6 supplies regression gate.
Prompt injection	P2	Implement/test where applicable; P6 supplies regression gate.
AI tool permissions	P2	Implement/test where applicable; P6 supplies regression gate.
Rate/request limits	P2	Implement/test where applicable; P6 supplies regression gate.
Secrets/TLS	P2	Implement/test where applicable; P6 supplies regression gate.
Audit logging	P2	Implement/test where applicable; P6 supplies regression gate.
Dependency/container scanning	P2	Implement/test where applicable; P6 supplies regression gate.

9. Observability

Required identifiers: trace_id, mission_id, run_id, job_id, organization_id, model_version, dataset_id. Metrics should expose latency, queue time, errors and user-visible mission state. Role metrics: agent_node_duration_ms, agent_tool_calls_total, confidence_gate_total, prompt_injection_block_total

10. Failure Modes

Failure	Detection	Recovery/fallback	User behavior
Bhoonidhi unavailable	timeout/5xx	bounded retry → pinned permitted fixture	show fallback clearly
Inference timeout/OOM	job timeout/worker exit	bounded retry → deterministic fixture for demo	show explicit degraded/failure state
Redis restart	connection/requeue failure	idempotent requeue/reconciliation	mission remains recoverable
Invalid AOI/raster	validator rejection	no expensive processing	show actionable validation
Low confidence	confidence gate	clarify/acquire/explicit uncertainty	never confident prose
Tenant violation	RBAC denial	no retry/no leakage	403/404

11. Implementation Order

Foundation → core domain → cross-service integration → differentiation → hardening/release. Earlier issues unlock later ones, but frozen contracts and fixtures allow parallel implementation. Do not serialize on live dependencies. Use mocks whenever the producer is unfinished.

12. Daily Execution Plan — Sep 11–20

Date	Issues	Expected output	Dependency	Integration checkpoint	Tests/demo
Sep 11	P2-01, P2-02	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 12	P2-02, P2-03	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 13	P2-03, P2-04	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 14	P2-04, P2-05	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 15	P2-05, P2-06	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 16	P2-06, P2-07	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 17	P2-07, P2-08	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state

Date	Issues	Expected output	Dependency	Integration checkpoint	Tests/demo
Sep 18	P2-08, P2-09	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 19	P2-09, P2-10	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 20	P2-10, P2-11	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state

13. Definition of Ready / Done

Field	Implementation specification
DoR	Requirements clear; owner/reviewer; contract; dependencies; acceptance; tests; demo result defined.
DoD	Merged; unit/contract/integration tests; security; telemetry; docs/schema; DEV/DEMO; acceptance demonstrated.

14. Cross-Team Communication

- P1 is architecture/contract authority.
- P6 is CI/DEV/DEMO/release authority.
- P2↔P3: sensor arbitration, inference and evidence.
- P2↔P4: observation selection, data quality and acquisition.
- P3↔P4: AssetRef, CRS/resolution and preprocessing.
- P4↔P5: tiles, masks, GeoJSON and map performance.
- P5↔P1: generated client and browser DTOs.
- Escalate blockers after 30–60 minutes; use contract mocks/fixtures instead of waiting.

15. Prompt to Give to the Coding Agent

You are P2 implementation agent. Own LangGraph, Mission Copilot, intent, tool registry, sensor arbitration, Evidence Graph and confidence gate. The LLM is coordinator/synthesizer, never source of truth. Treat user text, metadata and model output as untrusted. Only allow registered scoped tools. Never invent observations/geometry/measurements. Low confidence must clarify/acquire/return explicit uncertainty. Add agent regression, injection, schema, unit and integration tests plus trace/tool audit.

16. Role Release Checklist

- [] All P0 issues merged or explicitly waived.
- [] Unit + contract + relevant integration tests green.
- [] Security controls and negative tests green.
- [] Trace/metrics/logging present.
- [] DEV/DEMO verified.
- [] Failure behavior demonstrated.
- [] Fixtures/model versions pinned where applicable.
- [] Documentation/ADR updated.
- [] Role-specific flagship demo path works.